Trees, Searching, and Scraping (HW7)

Welcome back to CS 2100!

Prof. Rasika Bhalerao

Poll: What's wrong with this recursive function?

```python
def find_max(nums: list[int], index: int = 0) -> int:
    """Returns the maximum of the numbers in nums, starting from the given index"""
    if index == len(nums) - 1:
        return nums[index]
    else:
        max_of_rest: int = find_max(nums, index)
        if nums[index] < max_of_rest:
            return max_of_rest
        else:
            return nums[index]
```

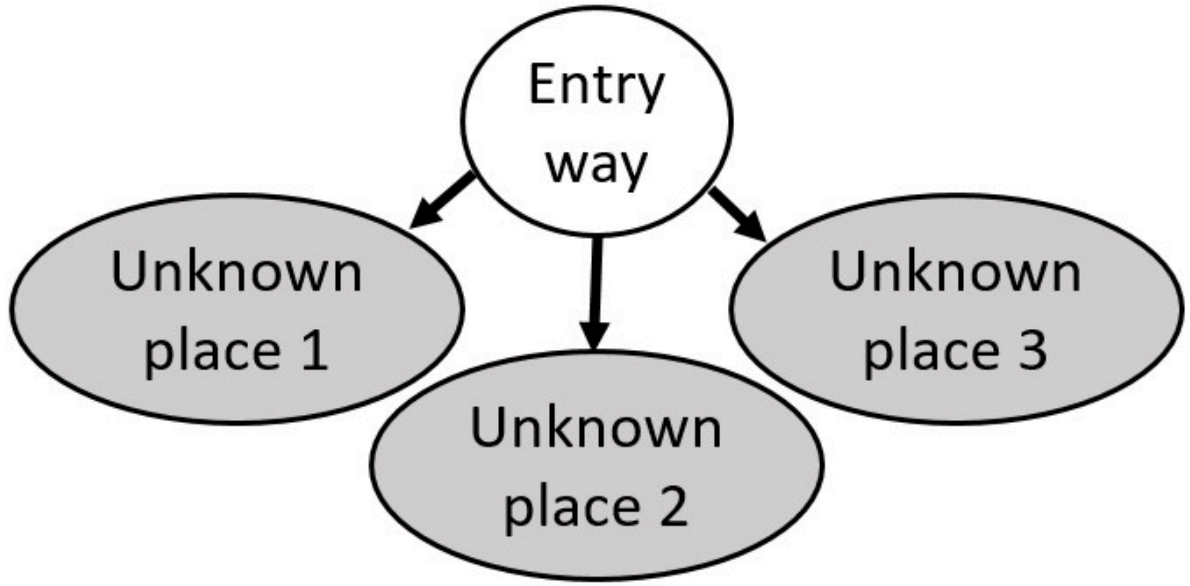
1. It's missing a base case
2. It's missing a recursive case
3. The recursive case doesn't progress towards the base case
4. It has an "off-by-one bug"

Poll: What's wrong with this recursive function?

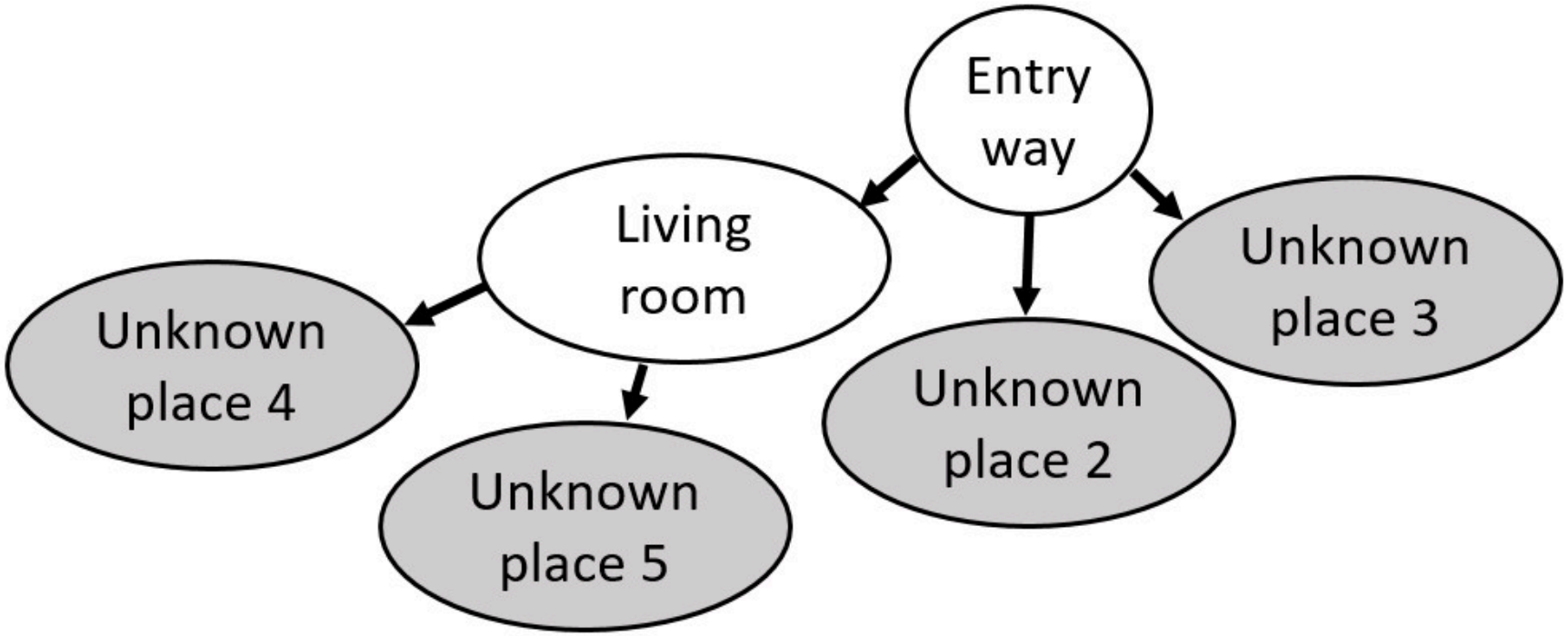
```python
def countdown(n: int) -> None:
    """Prints numbers from n down to 0, one on each line"""
    if n > 0:
        print(n)
        countdown(n - 1)
```

1. It's missing a base case
2. It's missing a recursive case
3. The recursive case doesn't progress towards the base case
4. It has an "off-by-one bug"

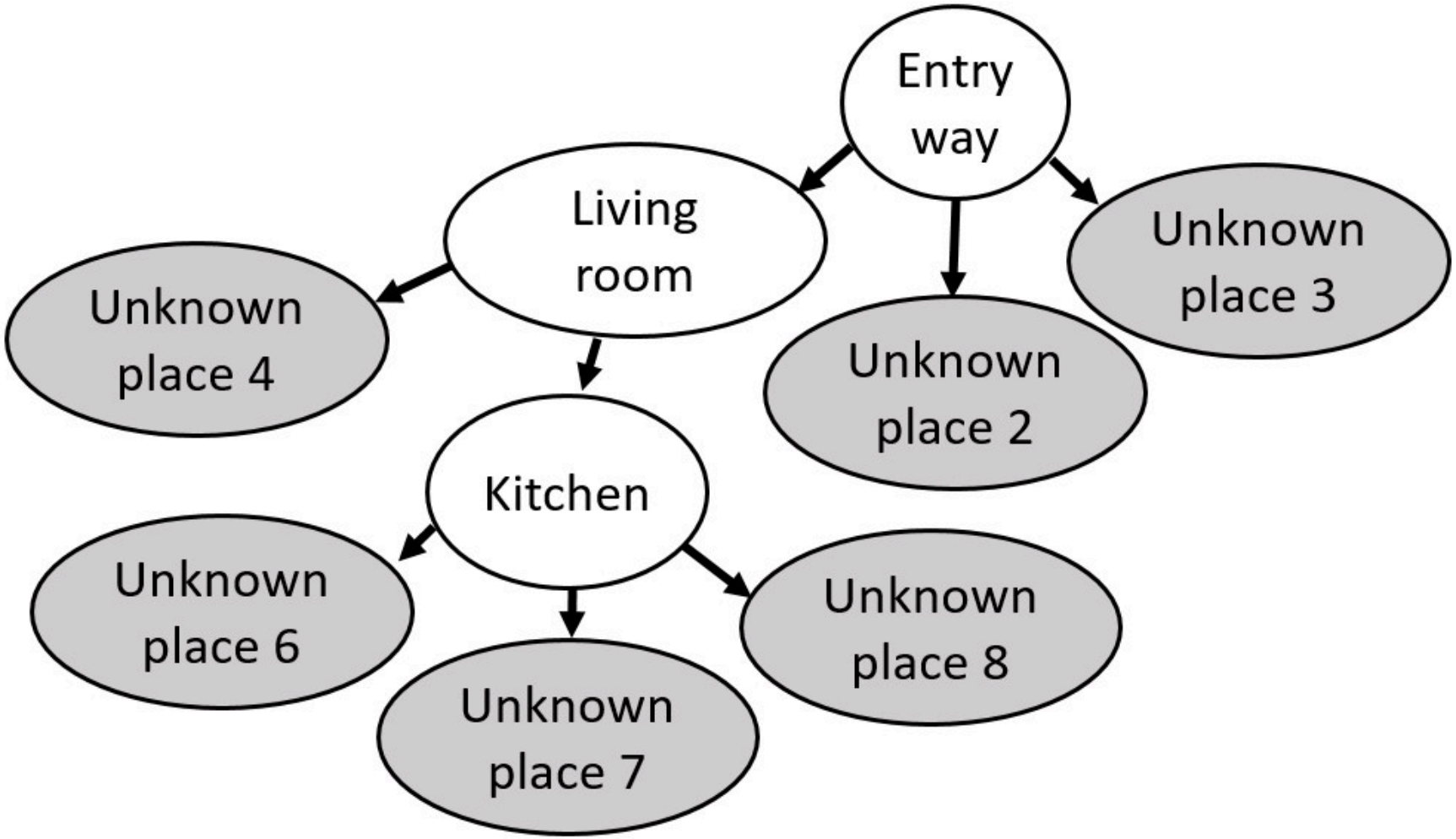
Let's say you're exploring a place...



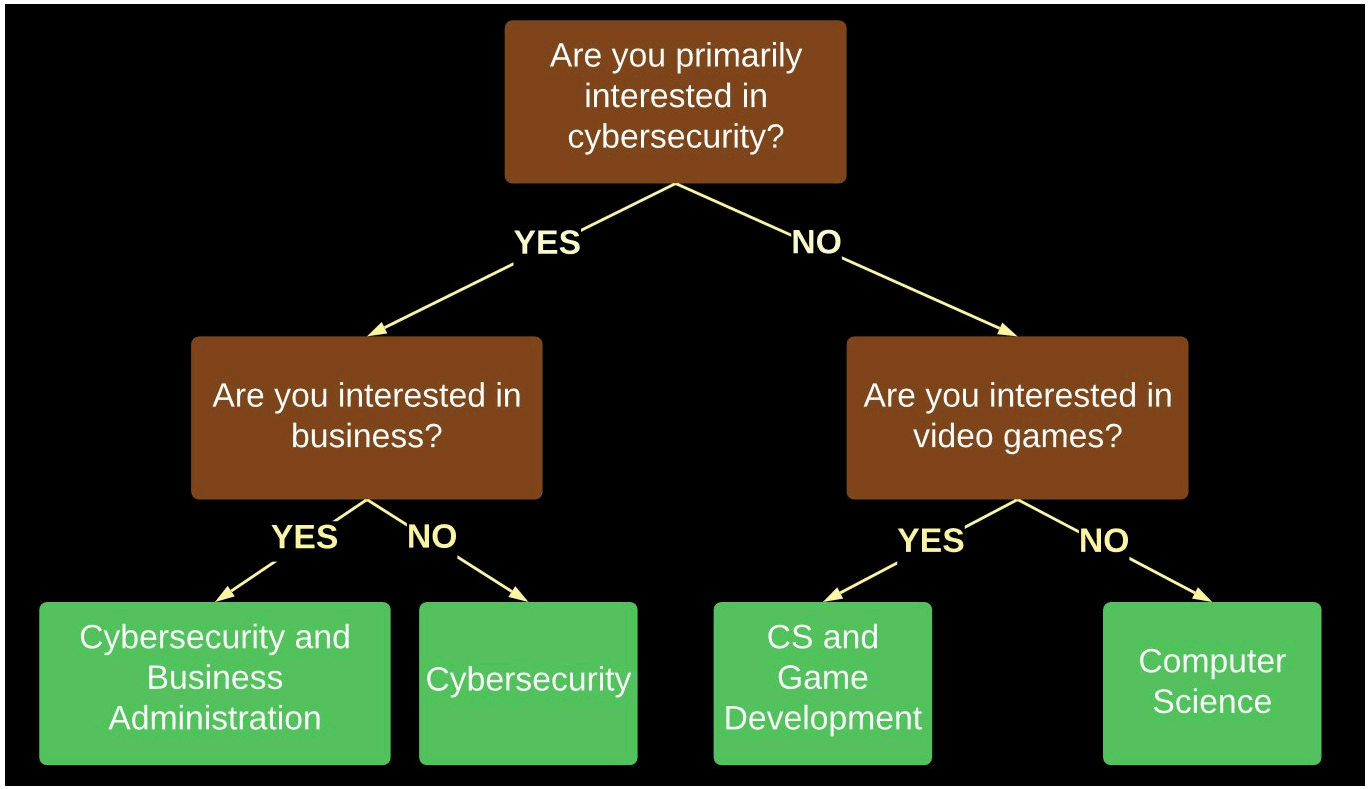
Let's say you're exploring a place...



Let's say you're exploring a place...



You may have seen trees like this decision tree:



Some tree terminology / rules

- Each *node* may have data
- There is a *root* node (the start)
- Each node points to any number of *child* nodes
- Cycles are not permitted
- There are any number of *leaf* nodes (node with no children)

Source: Reddit

Note: In computer science, we draw our trees with the root at the top and the leaves at the bottom.

Here is a recursive algorithm to search for an element in a `Tree`:

```
class Tree(Generic[T]):
    def __init__(self, root_data: Optional[T] = None) -> None:
        if root_data is None:
            self.root: Optional[Node[T]] = None
        else:
            self.root = Node[T](root_data)

    def __str__(self) -> str:
        return self.root.__str__()

    def __contains__(self, item: T) -> bool:
        return self.contains(item, self.root)

    def contains(self, item: T, node: Optional[Node[T]]) -> bool:
        if node is None:
            return False
        elif node.data == item:
            return True
        else:
            return self.contains(item, node.left) or self.contains(item, node.right)

tree: Tree[str] = Tree[str]('Entry way')

assert tree.root is not None
tree.root.left = Node[str]('Living room')

tree.root.left.right = Node[str]('Kitchen')

print('Kitchen' in tree)  # True
print('Bathroom' in tree)  # False
```

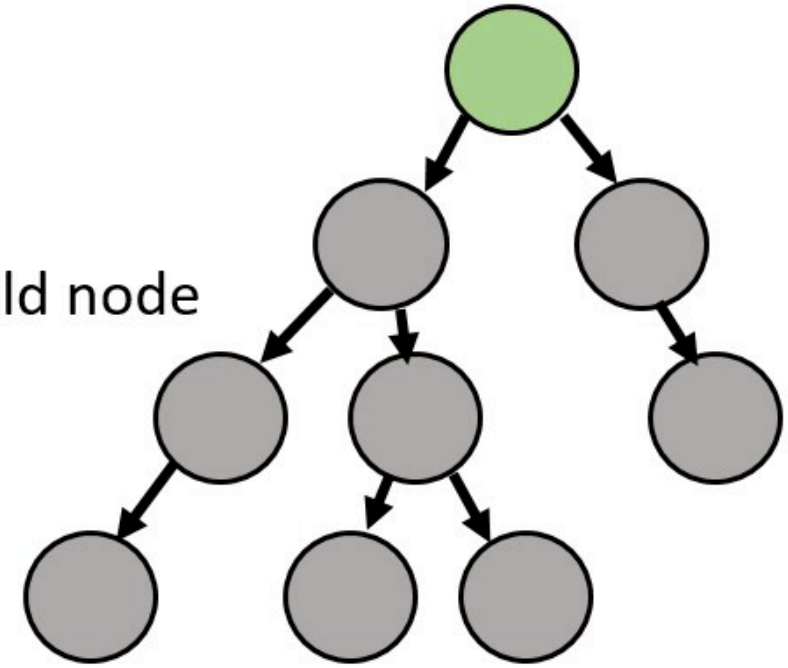
Let's modify it to work for trees with nodes that have any number of children, not just two.

Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

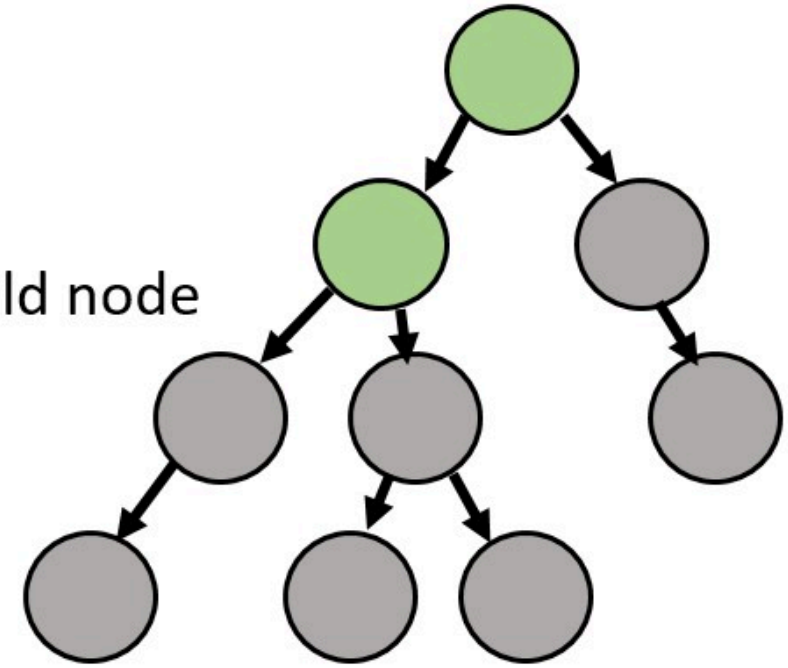


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

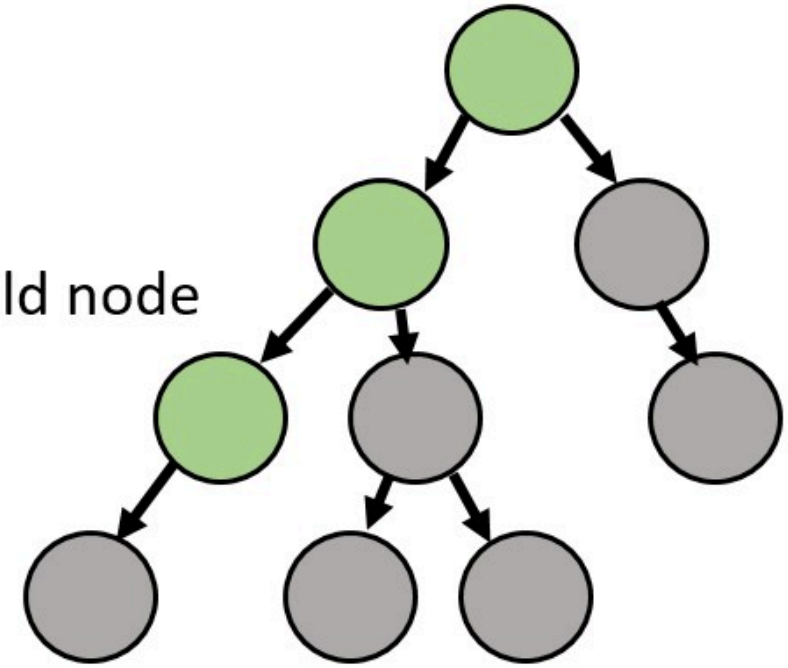


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

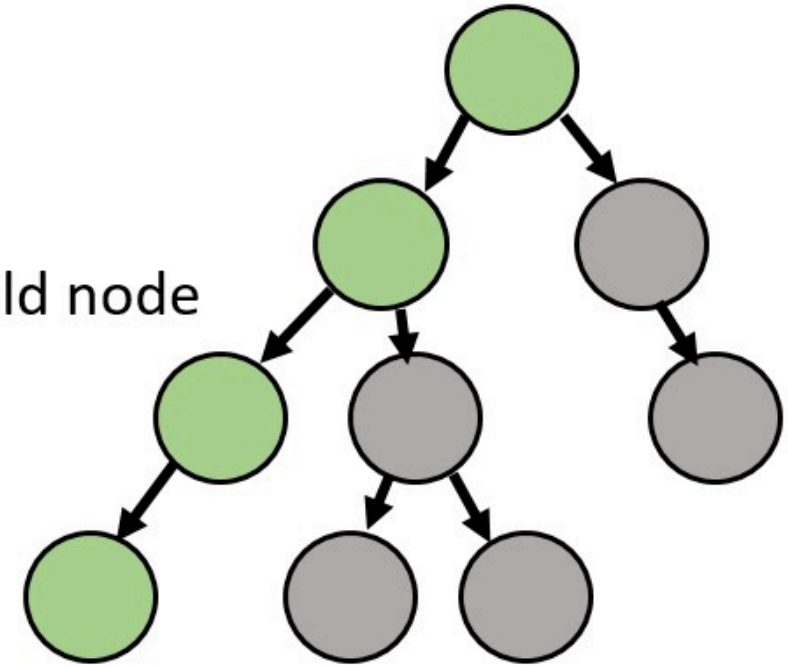


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

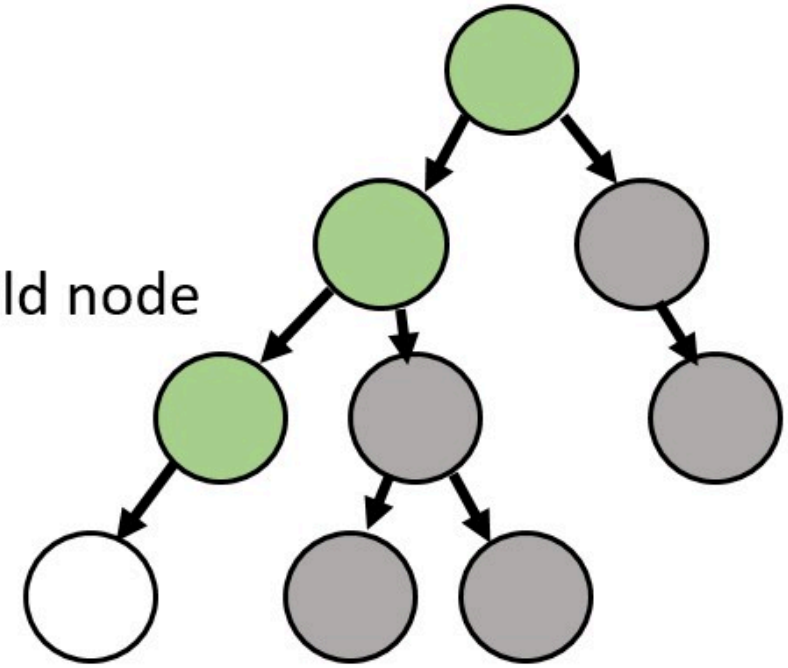


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

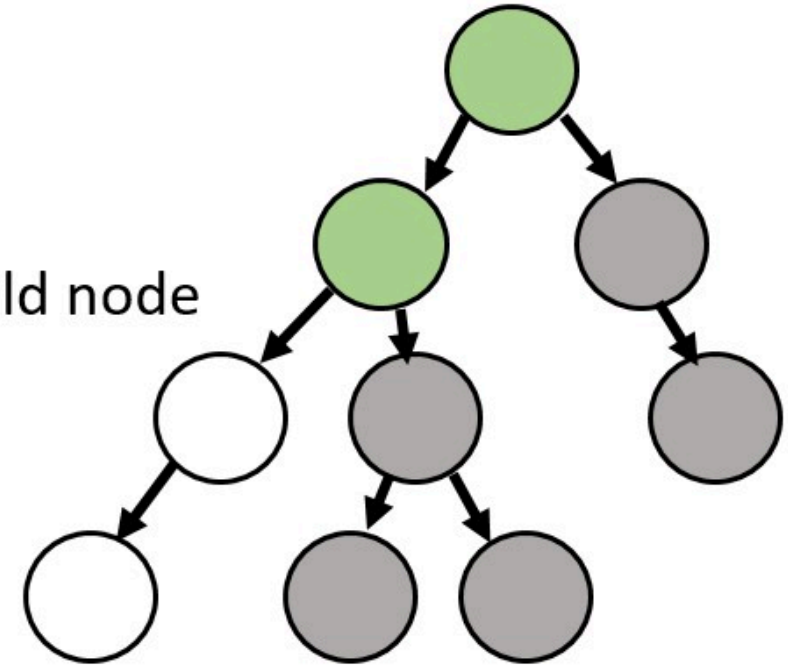


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

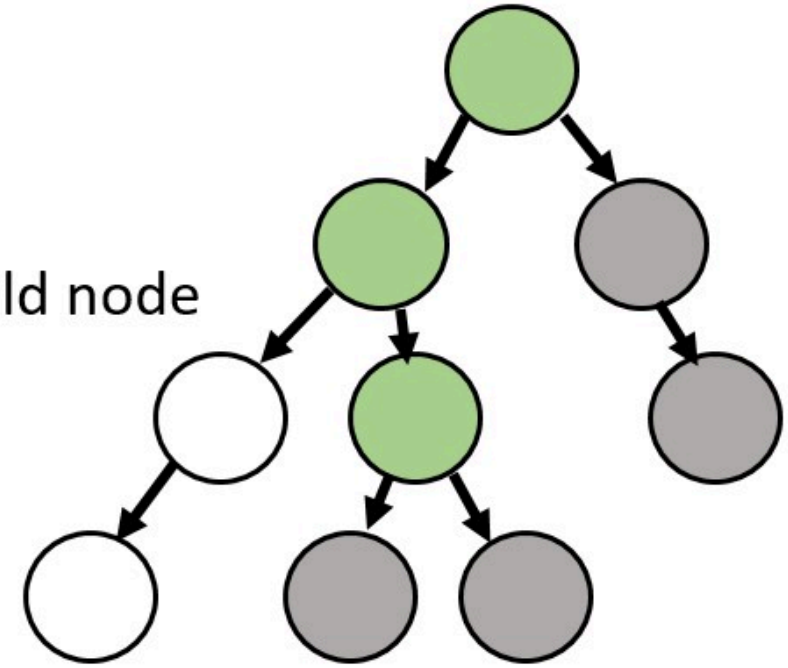


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

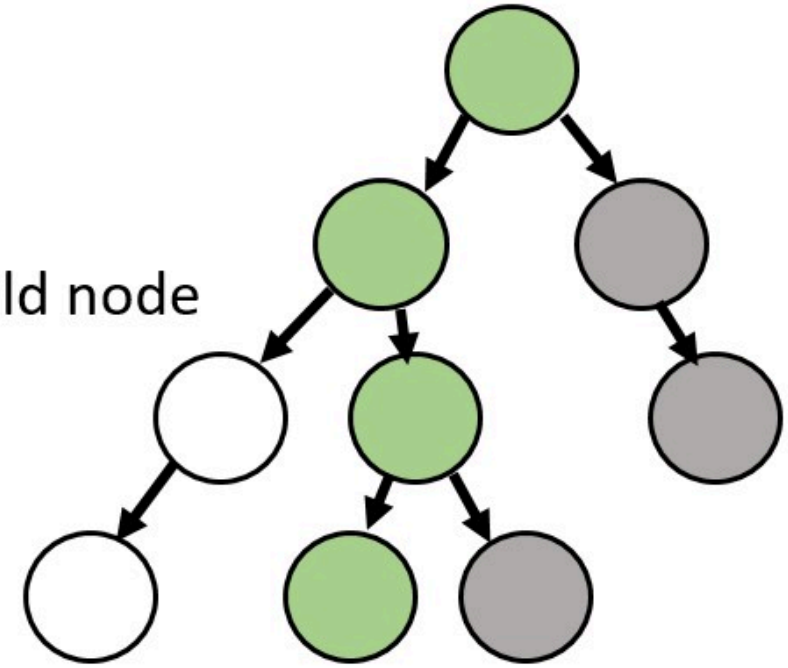


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

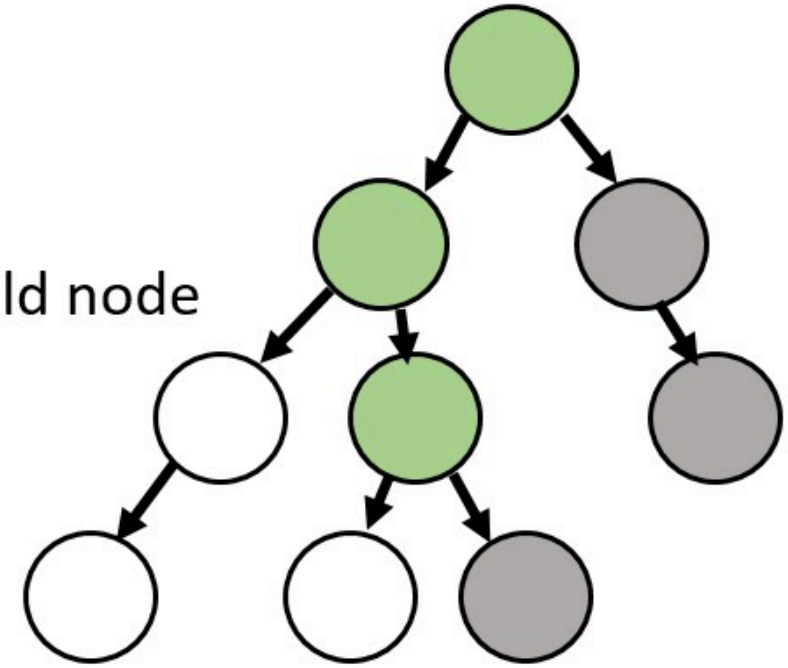


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

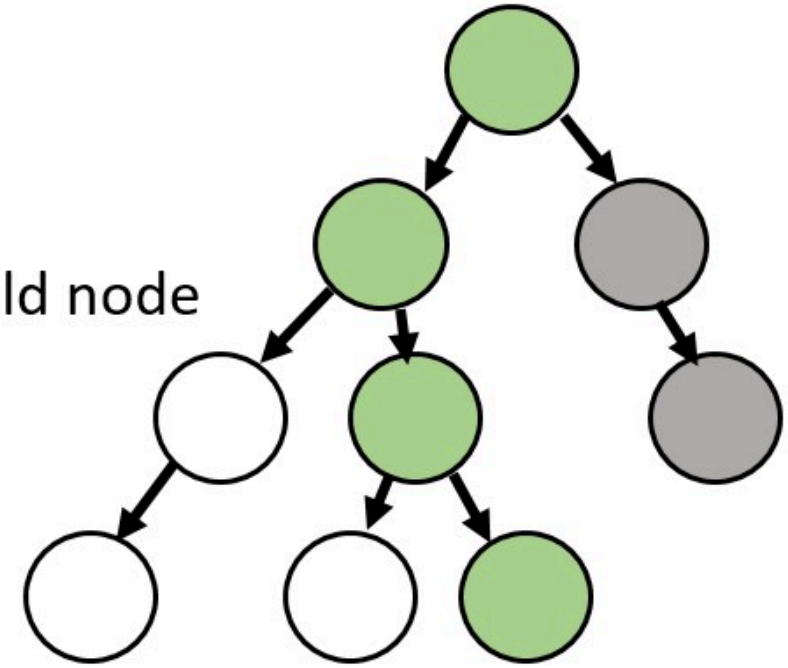


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

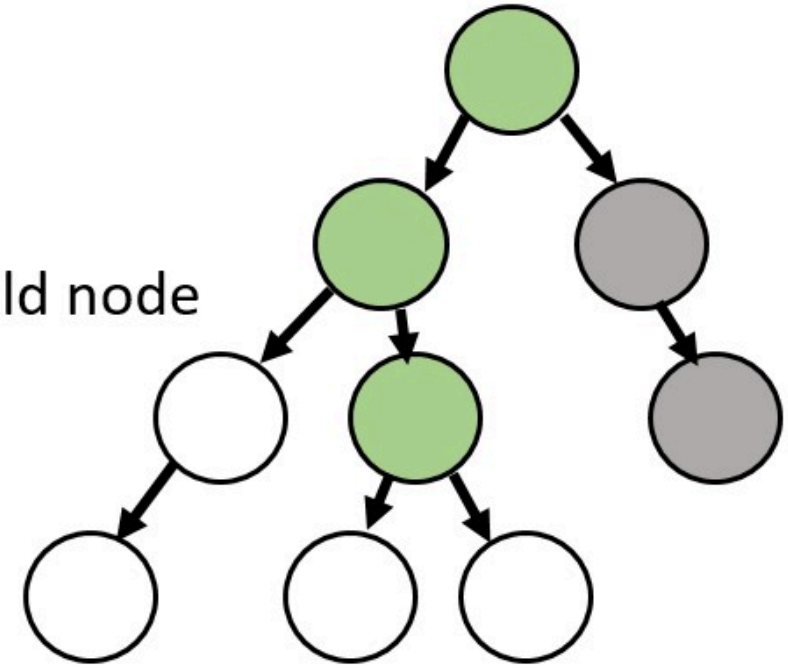


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

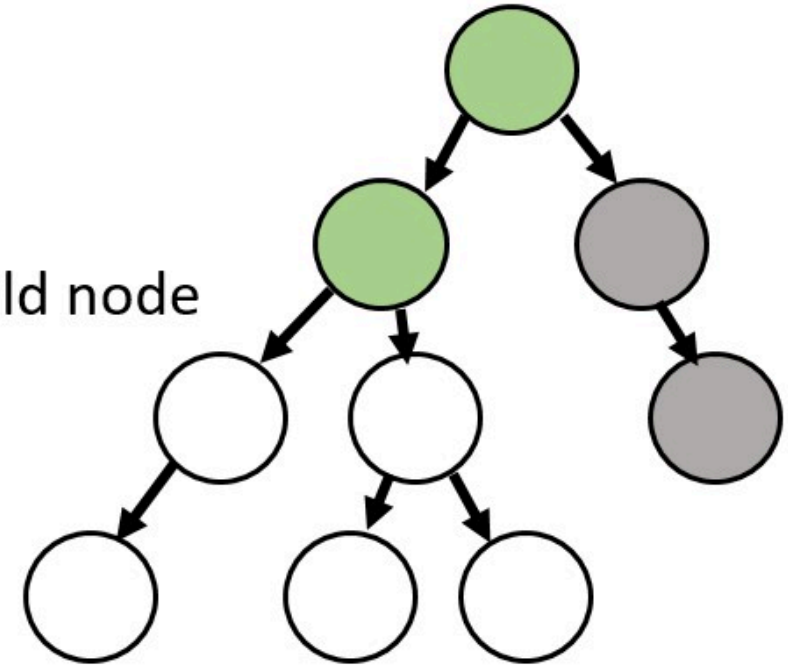


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

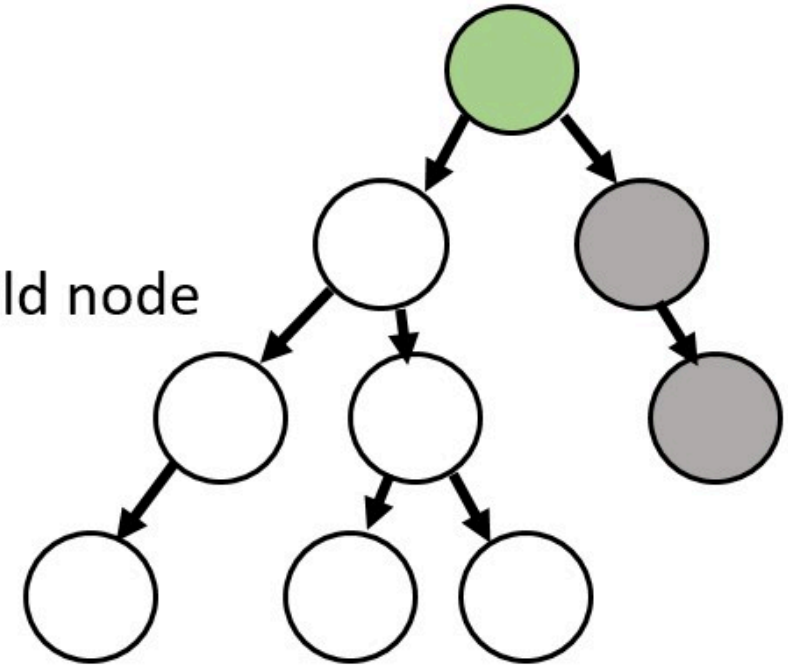


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

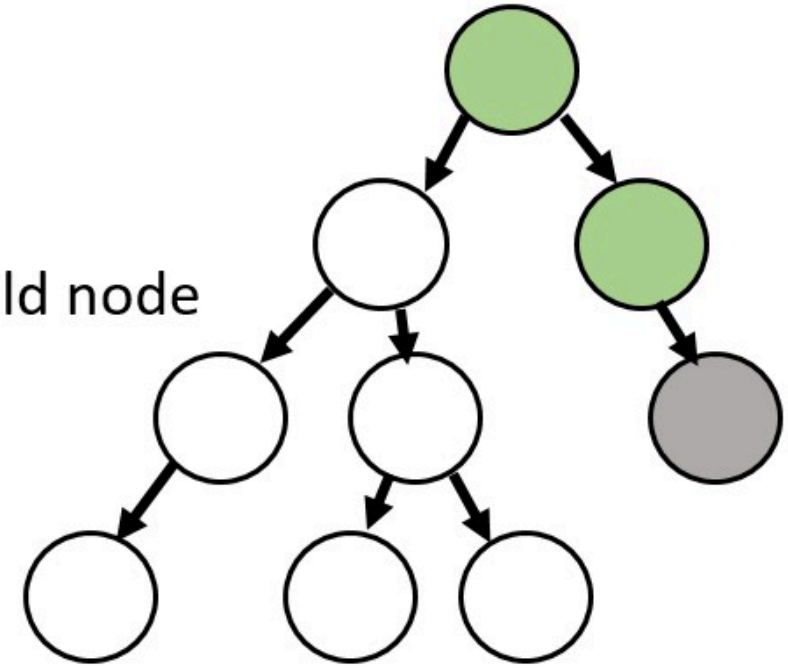


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

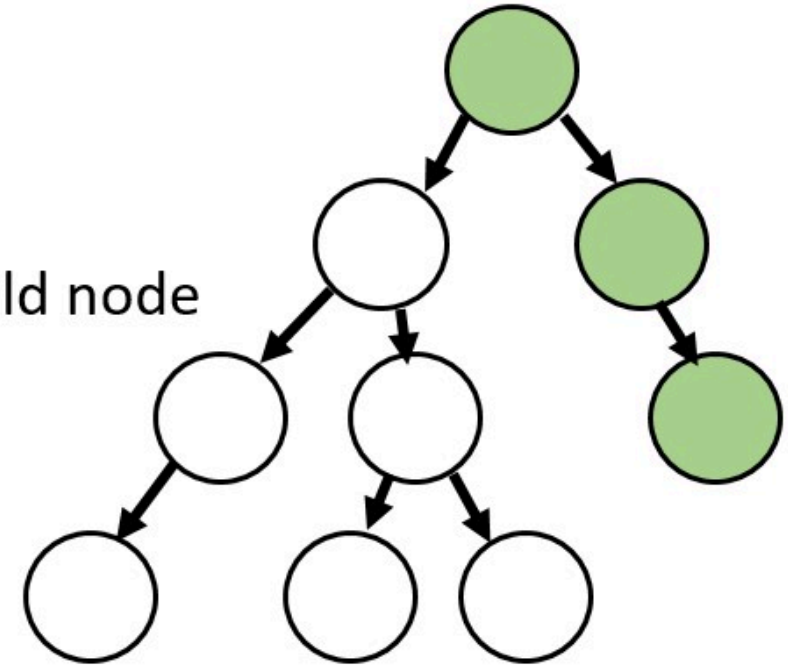


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

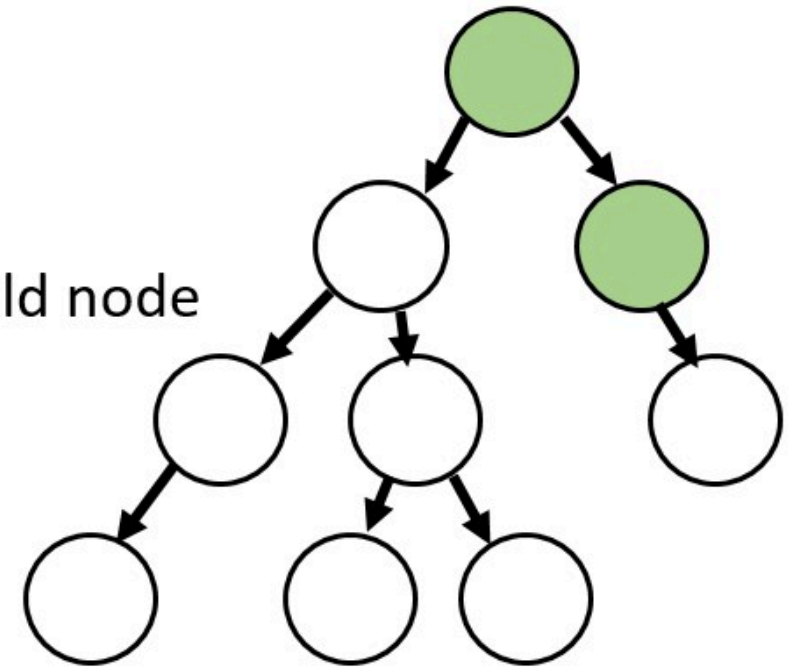


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

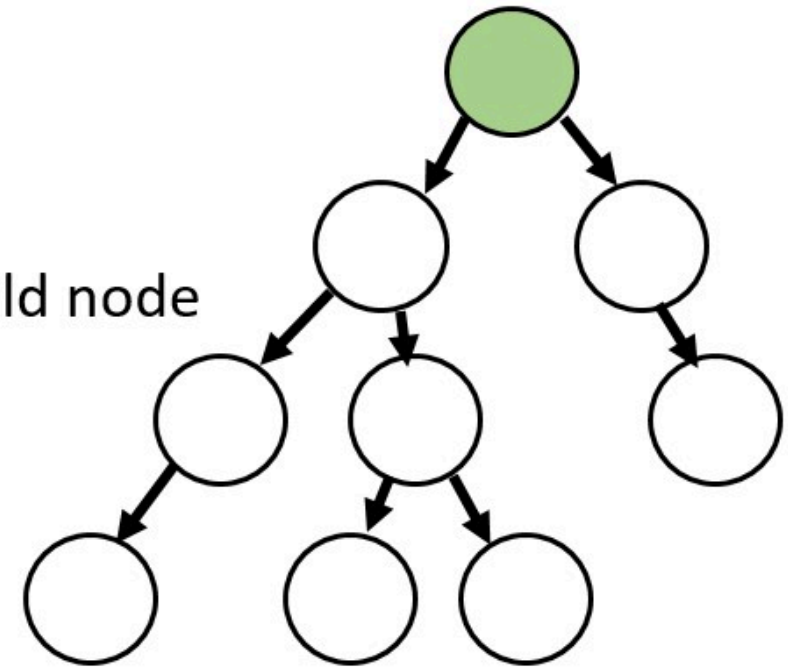


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child

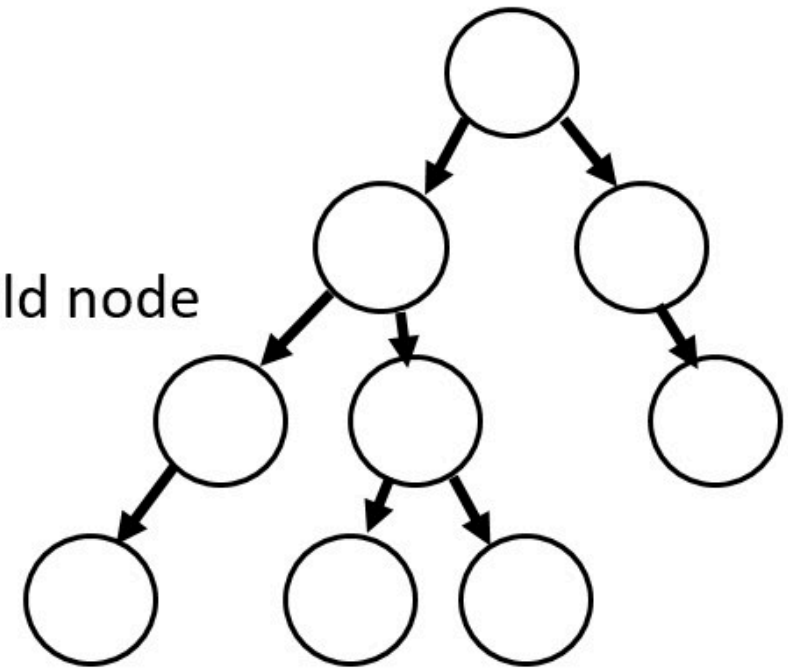


Recursive backtracking

Recursive backtracking is a strategy to search every node of a tree.

Strategy: given a node, for each of its children:

1. Choose a child to explore
2. Recursively perform this strategy for that child node
3. Un-choose that child



That recursive backtracking algorithm is called Depth-First Search, since it explores "deep" in one area before moving on to other unexplored "shallow" places (closer to the root).

Poll: This is pseudocode for a Depth-First Search on a graph that is not a tree (because it has cycles). What's a good base case?

```
DFS(node):
    Base case:
        ???
    Recursive case:
        For each child:
            Add child to explored nodes
            DFS(child)
```

1. If the node is in the set of explored nodes, do nothing
2. If the node is a leaf, add it to the set of explored nodes
3. If the node is `None`, do nothing
4. If the node is `None`, add it to the set of explored nodes

Scraping

Let's save this webpage to our computer: https://en.wikipedia.org/wiki/Web_scraping

(Webpage, Complete)

And let's look at the downloaded HTML file in VSCode.

It contains links to more pages, which contain links to more pages... it's a tree! (Or a graph because it has cycles)

Scraping webpages with BeautifulSoup

Instead of reading a website yourself, you can write code to download the page and extract the data you care about

```
import requests
from bs4 import BeautifulSoup

# 1. Fetch the webpage
response = requests.get("https://example.com")

# 2. Parse the HTML
soup = BeautifulSoup(response.text, "html.parser")

# 3. Find what you want
title = soup.find("h1")
links = soup.find_all("a")

# 4. Use the data!
print(title.text)
  ▶️ "Example Domain"
print(len(links), "links found")
  ▶️ 3 links found
```

Let's look at Homework 7 (Wikipedia Scraping)

Poll:

1. What is your main takeaway from today?

2. What would you like to revisit next time?